Lab 4

09/30/2026

Daniel Xu
ECE 3218L

1 Overview

Microcontrollers are digital, but many of the signals they must produce are analog. This lab uses the ATmega328P as a signal generator by driving an Analog Devices AD7302 digital-to-analog converter from the PORTD output pins. The programs made sawtooth, triangle, square, trapezoidal, and sine waveforms. The frequency of the waveforms was changed with a delay loop between samples or by skipping points of a lookup table.

2 Procedure

Every program writes one sample at a time to the DAC with the same sequence: write the byte to PORTD, pull PC0 low, and return it high, which latches the value into Channel A. PC1 is cleared once at reset to select Channel A, and PC0 starts high. Figure 1 shows the circuit on the lab board.

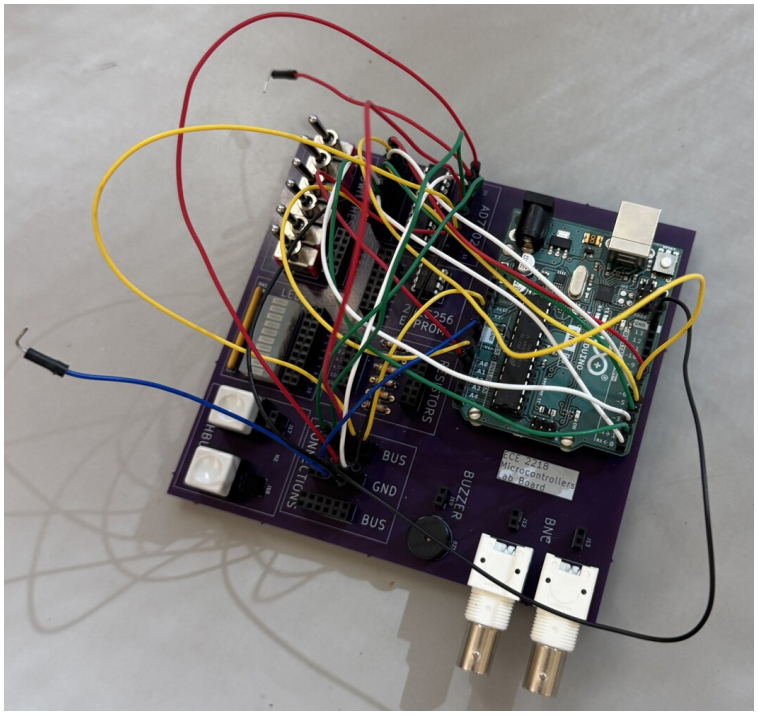


Figure 1: ATmega328P (Arduino Uno) wired to the AD7302 DAC on the lab board.

1. A program was written that loads the series 0x00, 0x01, . . . , 0xFF, 0x00, . . . into the DAC.

The sawtooth program increments a register after every sample and wraps from 0xFF to 0x00. The triangle program counts up to 0xFF and then back down to 0x00. The square wave reuses the triangle loop timing but outputs only 0x00 during the ascending half and 0xFF during the descending half, so it has the same frequency as the triangle wave.

2. A program was written to generate a trapezoidal waveform.

   The trapezoid counts up from 0x00 to 0xFF, holds 0xFF for a programmable number of samples, counts back down to 0x00, and holds 0x00 for the same number of samples. The constants TOP_LENGTH and BOT_LENGTH set the number of cycles to hold at the top and bottom of the trapezoid.

3. A program was written to generate a sine wave from a table.

   A table was generated with the values from the following equation:

$$s_k = \text{round}\left(127 \sin \frac{\pi k}{32}\right), \qquad k = 0, 1, \ldots, 31, \tag{1}$$

   This table contains 32 values. The minimum value is at $k = 0$, where $s_k = 0$. The maximum value is at $k = 16$, where $s_k = 127$. By using the 128 as the center of the DAC output, a sine wave can be generated with just these s_k values. The first half of the wave is generated with $128 + s_k$ and the second (negative) half is generated with $127 - s_k$.

4. A program was written to generate a triangle wave of variable frequency.

   By using a delay loop between sample outputs, the frequency of the triangle wave was able to be varied.

5. A program was written to vary the frequency of the sine wave by skipping table points.

   By incrementing the table index by more than 1, steps in the table are skipped and the frequency of the sine wave increased.

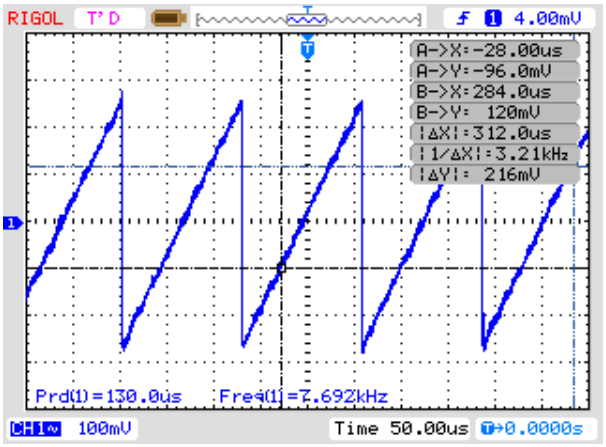
Table 1: Table of generated sine values from $s_k = \text{round}(127 \sin(\pi k/32))$.

k	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
s_k	0	12	25	37	49	60	71	81	90	98	106	112	117	122	125	126
k	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
s_k	127	126	125	122	117	112	106	98	90	81	71	60	49	37	25	12

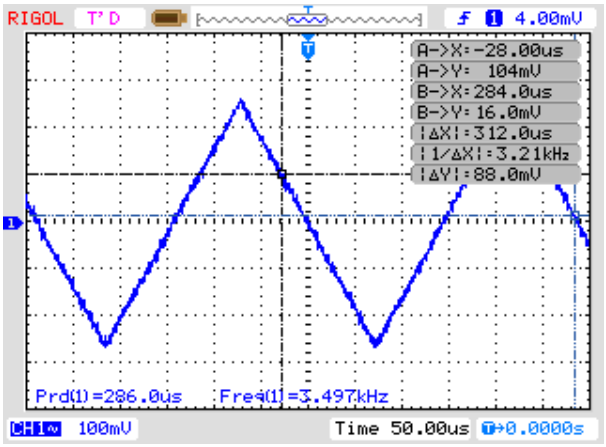
3 Discussion

3.1 Output Range

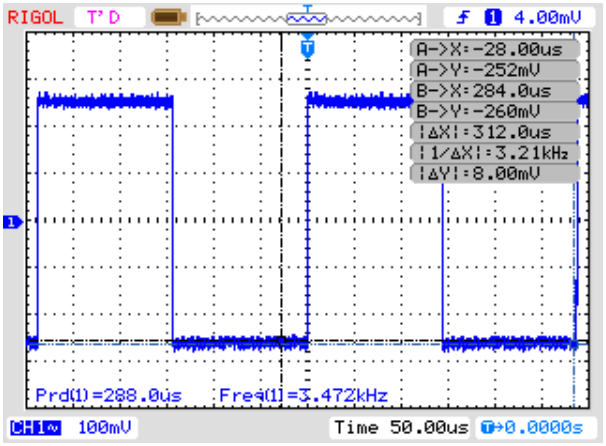
Writing 0x00 and then 0xFF to PORTD gave a maximum output of $V_{\text{max}} = 250$ mV. V_{max} is set by the reference voltage on REFIN, which this circuit ties to the supply V_{CC}. A change in the supply or the reference scales the maximum output voltage and the waveforms by the same factor.



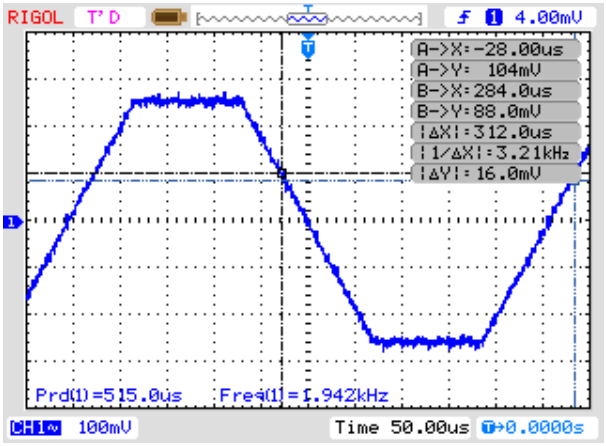
(a) Sawtooth: 130 μs, 7.69 kHz.



(b) Triangle: 286 μs, 3.50 kHz.



(c) Square: 288 μs, 3.47 kHz.



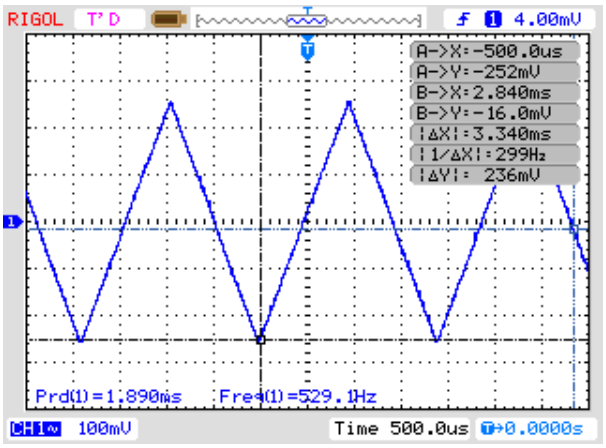
(d) Trapezoid: 515 μs, 1.94 kHz.

Figure 2: Sawtooth, triangle, square, and trapezoidal waves.

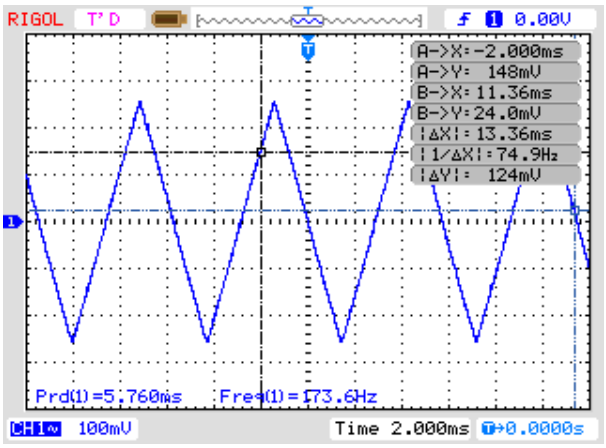
3.2 Measured Waveforms

All waveforms showed no DC offset on the oscilloscope and spanned ±250 mV (500 mV peak to peak). This was not expected, as the DAC should output voltages from 0 V to 5 V. Settings on the oscilloscope like AC coupling may have affected the measurements shown, resulting in a peak-to-peak voltage of 500 mV centered at 0V.

Figures 2–4 show the oscilloscope images, and Table 2 lists the periods and frequencies the oscilloscope measured.

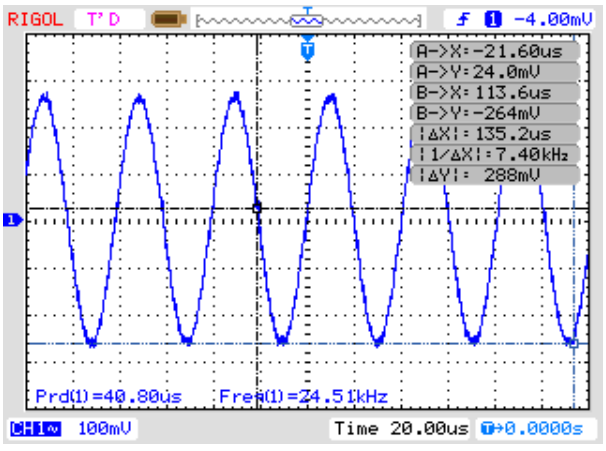


(a) Delay count 10: 1.890 ms, 529 Hz.

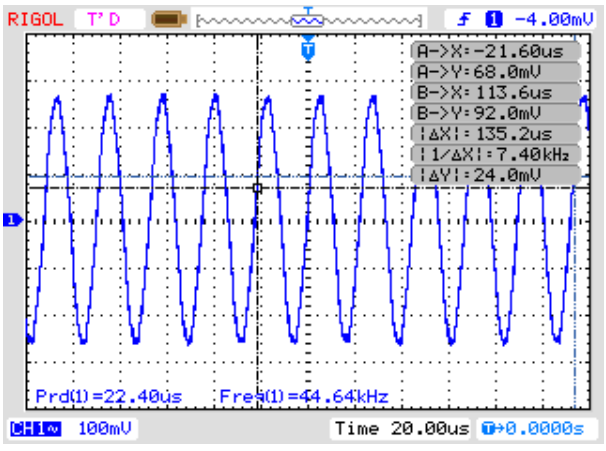


(b) Delay count 50: 5.760 ms, 174 Hz.

Figure 3: Triangle wave with a variable delay loop.



(a) Sine, every point: 40.8 μs, 24.5 kHz.



(b) Sine, every second point: 22.4 μs, 44.6 kHz.

Figure 4: Table-based sine wave.

Table 2: Measured waveform timing.

Waveform	Period	Frequency
Sawtooth	130.0 μs	7.692 kHz
Triangle	286.0 μs	3.497 kHz
Square	288.0 μs	3.472 kHz
Trapezoid	515.0 μs	1.942 kHz
Triangle, delay 10	1.890 ms	529.1 Hz
Triangle, delay 50	5.760 ms	173.6 Hz
Sine, step 1	40.80 μs	24.51 kHz
Sine, step 2	22.40 μs	44.64 kHz

The waveform shapes match the predictions. At 500 mV peak-to-peak, one DAC step is only about 2 mV, so the waveforms look smooth.

Note that skipping every other point halves the samples per period, but the period drops from 40.8 μs to 22.4 μs rather than to 20.4 μs. The frequency rises by a factor of 1.82 instead of 2, since each run of the skipping loop takes about one cycle longer.

3.3 Frequency Control with a Delay Loop

In Program 3, each sample takes $9+3N$ cycles, where N is the delay count: 9 cycles for the output and loop control, and $3N-1$ cycles for the countdown loop plus 1 cycle for the mov that reloads it. One period of the triangle wave has 510 samples, so at 16 MHz the predicted period is

$$T = \frac{510\,(9+3N)}{16\times 10^{6}\ \text{Hz}}. \tag{2}$$

Raising the count from 10 to 50 lengthened the period by

$$\Delta T_{\text{measured}} = \frac{5.760\ \text{ms} - 1.890\ \text{ms}}{50-10} = 96.8\ \mu\text{s} \tag{3}$$

per count, within about 1% of the predicted $510\cdot 3/16$ MHz $= 95.6$ μs. So each extra count adds the expected 3 cycles per sample.

4 Conclusion

In this lab, the ATmega328P drove an AD7302 DAC through PORTD. The programs generated sawtooth, triangle, square, trapezoidal, and sine waves, all with a span of 500 mV and no DC offset on the oscilloscope. The shape of each waveform matched the prediction. A countdown delay loop between samples lengthened the triangle period by 96.8 μs per count, within about 1% of the 95.6 μs predicted from the cycle count. The other waveforms also matched their cycle counts and expected periods.

A Full Code

Code 1: Program 1. Sawtooth, triangle, square, and trapezoidal waves.

```
.macro DACOUT                    ; DACOUT reg : write reg to DAC
   channel A
    out  PORTD, @0
    cbi  PORTC, 0
    sbi  PORTC, 0
.endmacro

.equ TOP_LENGTH = 200
.equ BOT_LENGTH = 200

.cseg
.org 0x0000
    rjmp reset

reset:
    ldi  r16, high(RAMEND)
    out  SPH, r16
    ldi  r16, low(RAMEND)
    out  SPL, r16

    sbi  PORTC, 0           ; PC0 = 1
    cbi  PORTC, 1           ; PC1 = 0 -> channel A (done once)
    sbi  DDRC, 0            ; PC0 output (goes out high, no glitch)
    sbi  DDRC, 1            ; PC1 output

    ldi  r16, 0xFF
    out  DDRD, r16          ; PD7-PD0 = data bus outputs
    clr  r17
    rjmp trapezoid

saw:
    DACOUT r17              ; 1 + 2 + 2 = 5
    inc  r17                ; 1
    rjmp saw                ; 2          -> 8 cycles/step

triangle:
tri_up:                        ; 0x00 to 0xFE
    DACOUT r17
    inc  r17
    cpi  r17, 0xFF
    brne tri_up
tri_down:                      ; 0xFF to 0x01
    DACOUT r17
    dec  r17
    nop
    brne tri_down
    rjmp tri_up

square:
    ldi  r18, 0x00          ; low level
    ldi  r19, 0xFF          ; high level
sq_up:
    DACOUT r18
    inc  r17
```

```
    cpi  r17, 0xFF
    brne sq_up
sq_down:
    DACOUT r19
    dec  r17
    nop
    brne sq_down
    rjmp sq_up

trapezoid:
    ldi r21, TOP_LENGTH
    ldi r22, BOT_LENGTH
    clr r17
    ; all sections should sum to 9 clock cycles

trap_bot_init:
    mov r20, r22 ; load bottom length

trap_bot:
    DACOUT r17
    nop
    dec r20
    brne trap_bot

trap_up:
    inc r17
    DACOUT r17
    cpi r17, 0xFE
    brne trap_up

    inc r17
    mov r20, r21

trap_top:
    DACOUT r17
    nop
    dec r20
    brne trap_top

trap_down:
    dec r17
    DACOUT r17
    cpi r17, 0x01
    brne trap_down

    dec r17
    rjmp trap_bot_init
```

Code 2: Program 2. Table-based sine wave with a fixed or variable table step.

```
.macro DACOUT                    ; DACOUT reg : write reg to DAC
   channel A
    out  PORTD, @0
    cbi  PORTC, 0
```

```
    sbi   PORTC, 0
.endmacro

.equ SKIP = 2                    ; table step per sample (1 to 31)

.cseg
.org 0x0000
    rjmp reset

reset:
    ldi   r16, high(RAMEND)
    out   SPH, r16
    ldi   r16, low(RAMEND)
    out   SPL, r16

    sbi   PORTC, 0           ; PC0 = 1
    cbi   PORTC, 1           ; PC1 = 0 -> channel A (done once)
    sbi   DDRC, 0            ; PC0 output (goes out high, no glitch)
    sbi   DDRC, 1            ; PC1 output

    ldi   r16, 0xFF
    out   DDRD, r16          ; PD7-PD0 = data bus outputs
    clr   r17
    rjmp sine_skip

sine:
    ldi   r19, 0x80          ; XOR mask: 0x80 = positive half, 0x7F =
       negative half
    ldi   r24, 0x00          ; WR low  (PC1 = 0 -> channel A)
    ldi   r25, 0x01          ; WR high (PC1 = 0 -> channel A)

sine_half:
    ldi   ZL, low(sine_table*2)    ; Z -> start of table (byte address)
    ldi   ZH, high(sine_table*2)
    ldi   r18, 32                ; 32 samples per half-cycle

sine_loop:
    lpm   r17, Z+            ; 3  fetch table vlaue, advance pointer
    eor   r17, r19           ; 1  -> 128+T or 127-T
    out   PORTD, r17         ; 1  data onto bus
    out   PORTC, r24         ; 1  WR low
    out   PORTC, r25         ; 1  WR high -> latched
    dec   r18                ; 1
    brne sine_loop           ; 2, 10 cycles/sample

    com   r19                ; swap halves: 0x80 <-> 0x7F
    rjmp sine_half

sine_skip:
        ldi   r19, 0x80          ; XOR mask: positive half
        ldi   r24, 0x00          ; WR low,  channel A
        ldi   r25, 0x01          ; WR high, channel A
        ldi   r22, SKIP          ; load step size
        ldi   ZH, high(sine_table*2)
```

```
57          ldi  ZL, low(sine_table*2)    ; = 0x00 because the table is
               aligned
58
59  ss_loop:
60          lpm  r17, Z               ; 3  fetch table value
61          eor  r17, r19             ; 1  -> 128+T or 127-T
62          out  PORTD, r17           ; 1
63          out  PORTC, r24           ; 1  WR low
64          out  PORTC, r25           ; 1  WR high
65          add  ZL, r22              ; 1  index += K
66          cpi  ZL, 32               ; 1  past end of half-cycle?
67          brlo ss_loop              ; 2       -> 11 cycles/sample
68
69          subi ZL, 32               ; wrap index, keeping the remainder
70          com  r19                  ; switch halves: 0x80 <-> 0x7F
71          rjmp ss_loop              ; wrap samples take 14 cycles
72
73  .org 0x0200                       ; word 0x200 = byte 0x400 -> low byte
       0x00
74  sine_table:
75      .db   0,  12,  25,  37,  49,  60,  71,  81
76      .db  90,  98, 106, 112, 117, 122, 125, 126
77      .db 127, 126, 125, 122, 117, 112, 106,  98
78      .db  90,  81,  71,  60,  49,  37,  25,  12
```

Code 3: Program 3. Triangle wave with variable frequency.

```
1   .equ DELAY = 10                   ; delay count per step (1..255)
2
3   .macro DACOUT                     ; DACOUT reg : write reg to DAC
       channel A
4       out  PORTD, @0
5       cbi  PORTC, 0
6       sbi  PORTC, 0
7   .endmacro
8
9   .equ TOP_LENGTH = 3
10  .equ BOT_LENGTH = 3
11
12  .cseg
13  .org 0x0000
14      rjmp reset
15
16  reset:
17      ldi  r16, high(RAMEND)
18      out  SPH, r16
19      ldi  r16, low(RAMEND)
20      out  SPL, r16
21
22      sbi  PORTC, 0             ; PC0 = 1
23      cbi  PORTC, 1             ; PC1 = 0 -> channel A (done once)
24      sbi  DDRC, 0              ; PC0 output (goes out high, no glitch)
25      sbi  DDRC, 1              ; PC1 output
26
```

```
27      ldi   r16, 0xFF
28      out   DDRD, r16           ; PD7 to PD0 = data bus outputs
29      clr   r17
30      rjmp triangle
31
32  triangle:
33          ldi   r21, DELAY          ; delay count (change this to change
                frequency)
34          clr   r17
35
36  triangle_up:                          ; 0x00 -> 0xFE
37          DACOUT r17                ; 5
38          mov   r20, r21            ; 1
39  triangle_d1:
40          dec   r20                 ; 1
41          brne triangle_d1          ; 2 (N iterations = 3N - 1 cycles)
42          inc   r17                 ; 1
43          cpi   r17, 0xFF           ; 1
44          brne triangle_up          ; 2 -> 9 + 3N cycles/step
45
46  triangle_down:                        ; 0xFF -> 0x01
47          DACOUT r17                ; 5
48          mov   r20, r21            ; 1
49  triangle_d2:
50          dec   r20                 ; 1
51          brne triangle_d2          ; 2 (3N - 1 cycles)
52          dec   r17                 ; 1
53          nop                       ; 1  pad to match 'up'
54          brne triangle_down        ; 2 -> 9 + 3N cycles/step
55          rjmp triangle_up
```